

Guía de Inicio: Configuración Inicial de la Aplicación Web

Esta guía te ayudará a preparar tu entorno local para empezar a construir la aplicación web de acceso único a videos. Utilizaremos **Flask (Python)** en el backend y **SQL puro (pg8000)** para interactuar con tu base de datos **PostgreSQL** de forma infalible en Windows y sin errores de compilación.

1. Estructura de Carpetas del Proyecto

Crea una carpeta principal en tu computadora (por ejemplo, **proyecto-videos**) y organiza los archivos de la siguiente manera:

```
proyecto-videos/  
■  
■■■ app.py # Archivo principal de tu servidor Flask  
■■■ database.sql # Script SQL para crear la tabla de la base de datos  
■■■ requirements.txt # Lista de dependencias de Python que necesitas instalar  
■■■ .env # Archivo secreto para guardar tus credenciales de la base de datos  
■  
■■■ templates/ # Carpeta para tus páginas HTML  
■ ■■■ login.html # Formulario de ingreso (Nombre y Apellido)  
■ ■■■ videos.html # Página que contendrá los 3 reproductores de video  
■  
■■■ static/ # Carpeta para archivos visuales y lógicos del navegador  
    ■■■ css/  
        ■ ■■■ style.css # Estilos visuales de la aplicación  
    ■■■ js/  
        ■■■ main.js # Código JavaScript para bloquear controles de video
```

2. Preparación del Entorno de Python

Para trabajar de forma ordenada, te recomiendo usar un **entorno virtual** de Python. Esto evita conflictos de librerías en tu computadora.

Pasos en tu Terminal/Consola:

1. Crea la carpeta de tu proyecto y entra en ella:

```
mkdir proyecto-videos
cd proyecto-videos
```

2. Crea el entorno virtual:

```
python -m venv venv
```

3. Activa el entorno virtual:

- **En Windows (CMD/PowerShell):**

```
venv\Scripts\activate
```

- **En macOS/Linux:**

```
source venv/bin/activate
```

3. Instalación de Dependencias

Crea un archivo llamado **requirements.txt** en la raíz de tu proyecto con el siguiente contenido:

```
Flask==3.0.3
pg8000==1.31.2
python-dotenv==1.0.1
```

Instala estas librerías ejecutando en tu terminal activada:

```
py -m pip install -r requirements.txt
```

- **Flask:** Nuestro framework para construir el servidor y las rutas web.
- **pg8000:** El conector de Python para comunicarse con PostgreSQL. Está escrito 100% en Python, por lo que **no requiere ningún compilador de C en Windows y nunca da errores de instalación.**
- **python-dotenv:** Nos permite separar las contraseñas de la base de datos del código fuente (muy importante antes de subir el proyecto a Render).

4. Estructura de la Base de Datos (`database.sql`)

Crea un archivo llamado **database.sql**. Este es el script que ejecutarás en tu panel de PostgreSQL (ya sea de forma local o en Render) para crear la tabla que registrará los usuarios de un solo uso:

```
-- Crear la tabla 'personas'
CREATE TABLE IF NOT EXISTS personas (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

*Nota: Agregamos **creado_en** para que se guarde automáticamente el día y la hora exacta en la que esa persona accedió a la plataforma, útil para tus controles de uso.*

5. El Esqueleto del Servidor (`app.py`)

Crea un archivo llamado **app.py**. Este es el punto de partida básico de nuestro backend en Flask:

```
import os
from flask import Flask, render_template, request, redirect, url_for, flash
from dotenv import load_dotenv

# Cargar variables de entorno desde el archivo .env
load_dotenv()

app = Flask(__name__)
app.secret_key = os.getenv("SECRET_KEY", "un_secreto_muy_seguro_temporal")

# Ruta 1: Pantalla de Login (Formulario)
@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        nombre = request.form.get("nombre", "").strip()
        apellido = request.form.get("apellido", "").strip()

        if not nombre or not apellido:
            flash("Por favor, completa ambos campos.", "danger")
            return redirect(url_for("login"))

        # TODO: Aquí programaremos la validación contra PostgreSQL en el siguiente paso.
        # Por ahora, simulamos un ingreso exitoso:
        return redirect(url_for("ver_videos"))

    return render_template("login.html")

# Ruta 2: Pantalla de Videos (Solo accesible tras validación)
@app.route("/videos")
def ver_videos():
    # TODO: Aquí validaremos que el usuario realmente acaba de ingresar
    return render_template("videos.html")

if __name__ == "__main__":
    app.run(debug=True)
```